
The Main Menu

When the program starts, you'll be greeted with a simple menu:

1.) First Come, First Serve 2.) Shortest Job First 3.) Shortest Remaining Time 4.) Round Robin 5.) Priority Scheduling 6.) Exit

Just type the number of whichever algorithm you want to try and hit **Enter**. Easy.

Before You Start Entering Data

There are a few things worth knowing before you dive in, just so the program doesn't catch you off guard:

- It only accepts **whole numbers** — no decimals, no letters
 - You need at least **3 processes** to run any algorithm
 - **Burst Time** and **Arrival Time** must be **0 or higher**
 - **Priority Values** range from **1 to however many processes you have**
 - The **Time Quantum** (used in Round Robin) must be **at least 2**
 - If you accidentally type something wrong, don't worry — the program will just ask you again
-

How Each Algorithm Works (Input-wise)

1. First Come, First Serve

This one's the most straightforward. The program will ask you how many processes you have, then for each one, it'll ask for the **Burst Time** (how long the process needs the CPU) and the **Arrival Time** (when it shows up). Once you've entered everything, it immediately shows you the results.

2. Shortest Job First

Same as FCFS — **Burst Time** and **Arrival Time** for each process. Behind the scenes though, the program will automatically pick whichever available process has the shortest burst time instead of just going in order of arrival.

3. Shortest Remaining Time

Again, same inputs — **Burst Time** and **Arrival Time**. The difference here is that this one is preemptive, meaning the program simulates the scheduling moment by moment. If a new process arrives with a shorter remaining time than whatever's currently running, it gets bumped to the front immediately.

4. Round Robin

This one has one extra step. After entering the **Burst Time** and **Arrival Time** for all your processes, the program will ask you for a **Time Quantum** — basically how many time units each process gets before it has to give way to the next one. Just remember it has to be at least 2.

5. Priority Scheduling

This one's a little different because once you pick it, you'll get a second menu:

Non-Preemptive Priority — Here you only need to enter the **Burst Time** and a **Priority Value** for each process. No arrival times needed since the program assumes everyone shows up at time 0. Whichever process has the highest priority (lowest number) goes first and runs all the way through without interruption.

Preemptive Priority — This time you'll also enter **Arrival Times**. If a higher-priority process arrives while another is running, the running process gets paused immediately and the new one takes over.

Priority + Round Robin — Same inputs as Preemptive Priority, plus a **Time Quantum** at the end. Processes are grouped by priority, but within the same priority level, they take turns fairly using Round Robin instead of one hogging the CPU the whole time.

Understanding the Output

Once you've entered everything, the program prints two things.

First, a **results table** that breaks down how each process did:

Column	What it means
Arrival Time	When the process arrived
Burst Time	How long it needed the CPU
Priority	Its priority level (only shows up in Priority Scheduling)
Waiting Time	How long it waited before getting the CPU
Turnaround Time	Total time from arrival to completion

Below the table, you'll also see the **average Waiting Time** and **average Turnaround Time** across all processes — useful for comparing how efficient the algorithm was.

Second, a **Gantt Chart** that visually shows you the order and timing of execution, like this:

Gantt Chart: | P1 | P2 | P5 | P4 | P3 |
 0 8 12 14 19 28

Each block is a process, and the numbers on the bottom are the time marks showing exactly when each one started and ended.

Running Multiple Algorithms

After each run, the program automatically brings you back to the main menu — so you can test as many algorithms as you want in one session without restarting. When you're done, just select **option 6** to exit.